PHASE 7 — Assets, Expenses, Announcements & Helpdesk

PRSECURITY HRMS

IMPORTANT RULES

Output every file 100% completely. Never use '...' or 'rest remains same'.
After each file write "--- FILE COMPLETE ---" then start the next file immediately.
If about to hit output limit, finish current file and write "--- PAUSED: Type CONTINUE ---"
This is Phase 7 of 8. Phases 1–6 done: Auth, Employees, Dashboards, Attendance, Leave, Payroll, Tasks, Projects, Clients, Recruitment, Performance.
DO NOT rebuild previous files. Only add NEW files or EXTEND where specified.

CONTEXT

Project: PRSECURITY HRMS | Stack: React+Vite+TS+Tailwind+shadcn/ui | Node+Express+Prisma+PostgreSQL Roles: ADMIN, HR, EMPLOYEE, INTERN | Currency: ₹

PART A — ASSETS & EXPENSES

STEP A1 — Update Prisma Schema

prisma

```prisma
model Asset {
  id           String      @id @default(uuid())
  name         String
  type         AssetType
  serialNumber String?     @unique
  assignedTo   String?
  assignedAt   DateTime?
  returnedAt   DateTime?
  status       AssetStatus @default(AVAILABLE)
  purchaseDate DateTime?
  purchasePrice Float?
  condition    String?
  notes        String?
  assignee     User?       @relation(fields: [assignedTo], references: [id])
  createdAt    DateTime    @default(now())
  updatedAt    DateTime    @updatedAt
}

enum AssetType { LAPTOP PHONE MONITOR PERIPHERAL VEHICLE OTHER }
enum AssetStatus { AVAILABLE ASSIGNED MAINTENANCE RETIRED }

model ExpenseClaim {
  id          String        @id @default(uuid())
  userId      String
  title       String
  amount      Float
  category    String
  receiptUrl  String?
  date        DateTime
  description String?
  status      ExpenseStatus @default(PENDING)
  reviewedBy  String?
  reviewedAt  DateTime?
  reviewerNote String?
  paidAt      DateTime?
  user        User          @relation(fields: [userId], references: [id])
  reviewer    User?         @relation("ExpenseReviewer", fields: [reviewedBy], references: [id])
  createdAt   DateTime      @default(now())
  updatedAt   DateTime      @updatedAt
}

enum ExpenseStatus { PENDING APPROVED REJECTED PAID }
```

Run: `npx prisma migrate dev --name add_assets_expenses`

STEP A2 — Assets Backend

FILE: backend/src/services/asset.service.ts

`getAllAssets(query)`:

Paginated, filter by type, status, assignedTo

Include assignee name

`createAsset(data)`:

Create asset (Admin/HR)

`updateAsset(id, data)`:

Update asset details

`assignAsset(assetId, userId)`:

Check asset status=AVAILABLE

Set assignedTo=userId, assignedAt=now, status=ASSIGNED

Notify employee: "Asset {name} (S/N: {serial}) has been assigned to you"

`returnAsset(assetId)`:

Set assignedTo=null, returnedAt=now, status=AVAILABLE

`getMyAssets(userId)`:

All assets currently assigned to userId

`getAssetStats()`:

Count by status: available, assigned, maintenance, retired

Total assets, total value (₹)

FILE: backend/src/controllers/asset.controller.ts

FILE: backend/src/routes/asset.routes.ts

```
GET   /api/assets          → getAllAssets (HR/Admin)
POST  /api/assets          → createAsset (HR/Admin)
GET   /api/assets/stats    → getAssetStats (HR/Admin)
GET   /api/assets/my       → getMyAssets (authenticated)
GET   /api/assets/:id      → getAsset (HR/Admin)
PUT   /api/assets/:id      → updateAsset (HR/Admin)
POST  /api/assets/:id/assign → assignAsset (HR/Admin)
POST  /api/assets/:id/return → returnAsset (HR/Admin)
```

STEP A3 — Expenses Backend

FILE: backend/src/services/expense.service.ts

`getMyExpenses(userId, query)`:

Paginated list, filter by status

`createExpenseClaim(userId, data, receiptFile?)`:

Create claim, save receipt via multer if provided

Notify HR/Admin

`getAllExpenses(query)`:

HR/Admin: all claims with user info, filter by status/userId/dateRange

`approveExpense(id, reviewerId, note?)`:

Update status=APPROVED

Notify employee

`rejectExpense(id, reviewerId, note)`:

Update status=REJECTED with note

Notify employee

`markExpensePaid(id)`:

Update status=PAID, paidAt=now

`getExpenseStats()`:

Total pending amount (₹), total approved this month (₹), count by status

FILE: backend/src/controllers/expense.controller.ts

FILE: backend/src/routes/expense.routes.ts

```
GET    /api/expenses          → getAllExpenses (HR/Admin)
GET    /api/expenses/my       → getMyExpenses (authenticated)
GET    /api/expenses/stats    → getExpenseStats (HR/Admin)
POST   /api/expenses          → createExpenseClaim (authenticated, multipart)
GET    /api/expenses/:id      → getExpense (authenticated)
PUT    /api/expenses/:id/approve → approveExpense (HR/Admin)
PUT    /api/expenses/:id/reject  → rejectExpense (HR/Admin)
PUT    /api/expenses/:id/paid    → markExpensePaid (HR/Admin)
```

STEP A4 — Assets & Expenses Frontend

FILE: frontend/src/api/asset.api.ts

FILE: frontend/src/api/expense.api.ts

FILE: frontend/src/pages/assets/AssetsPage.tsx

HR/Admin view:

PageHeader: "Asset Management" + "Add Asset" button

StatCards: Total Assets, Available, Assigned, In Maintenance

DataTable columns: Name, Type badge, Serial Number, Assigned To (avatar+name or "—"), Status badge, Purchase Date, Purchase Price (₹), Actions

Filters: Type, Status

Actions: Edit, Assign (if AVAILABLE), Return (if ASSIGNED), Update Status

Assign Asset: modal with employee dropdown, click confirm → assigns

FILE: frontend/src/components/assets/AssetFormModal.tsx

Create/Edit: Name, Type dropdown, Serial Number, Purchase Date, Purchase Price (₹), Condition, Notes.

FILE: frontend/src/pages/expenses/ExpensesPage.tsx

Two views based on role:

Employee/Intern view:

PageHeader: "My Expenses" + "Submit Claim" button

StatCards: Pending (₹), Approved (₹), Paid (₹)

Table: Title, Category, Amount (₹), Date, Status badge, Receipt (download link), Actions (cancel if pending)

"Submit Claim" → opens ExpenseFormModal

HR/Admin view:

PageHeader: "Expense Claims"

StatCards: Total Pending (₹), Total Approved This Month (₹), Claims to Review
Tabs:
Pending: table with approve/reject buttons per row
All Claims: full filterable table with export CSV
Approve: confirm modal with optional note
Reject: modal requiring note

FILE: frontend/src/components/expenses/ExpenseFormModal.tsx

Form: Title, Category (Travel/Food/Accommodation/Equipment/Other), Amount (₹), Date, Description, Receipt upload (image/PDF).

PART B — ANNOUNCEMENTS

STEP B1 — Update Prisma Schema

```prisma
model Announcement {
  id          String   @id @default(uuid())
  title       String
  content     String
  priority    AnnouncementPriority @default(LOW)
  targetRoles String[]
  postedBy    String
  expiresAt   DateTime?
  poster      User     @relation(fields: [postedBy], references: [id])
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

enum AnnouncementPriority { LOW MEDIUM HIGH URGENT }
```

Run: `npx prisma migrate dev --name add_announcements_helpdesk` (combine with helpdesk below in one migration)

STEP B2 — Announcements Backend

FILE: backend/src/services/announcement.service.ts

`getAnnouncements(userRole, query)`:
Return non-expired announcements where targetRoles contains userRole or targetRoles is empty (all roles)
URGENT pinned at top, then sorted by createdAt desc
Paginated

`createAnnouncement(data, postedBy)`:
data: title, content, priority, targetRoles (array), expiresAt
Notify all users in targetRoles via socket

`updateAnnouncement(id, data)`:
Update fields

`deleteAnnouncement(id)`:
Hard delete (HR/Admin)

FILE: backend/src/controllers/announcement.controller.ts

FILE: backend/src/routes/announcement.routes.ts

```
GET    /api/announcements      → getAnnouncements (authenticated)
POST   /api/announcements      → createAnnouncement (HR/Admin)
PUT    /api/announcements/:id  → updateAnnouncement (HR/Admin)
DELETE /api/announcements/:id  → deleteAnnouncement (HR/Admin)
```

STEP B3 — Announcements Frontend

FILE: frontend/src/api/announcement.api.ts

FILE: frontend/src/pages/announcements/AnnouncementsPage.tsx

All roles view:
PageHeader: "Announcements" + "Post Announcement" button (HR/Admin only)
URGENT announcements: pinned section at top with red left border card
Rest: card feed sorted by date
Each card: priority badge, title, content (full), posted by (avatar+name), posted date, expiry date if set
HR/Admin: Edit + Delete buttons on each card
"Post Announcement" button → opens AnnouncementFormModal

FILE: frontend/src/components/announcements/AnnouncementFormModal.tsx

Form: Title, Content (textarea), Priority dropdown, Target Roles (multi-checkbox: ALL/ADMIN/HR/EMPLOYEE/INTERN), Expires At (optional date).

Update frontend/src/pages/dashboard/EmployeeDashboardPage.tsx

Replace announcements EmptyState with real AnnouncementsPage data (latest 3 announcements).

PART C — HELPDESK

STEP C1 — Update Prisma Schema (same migration as announcements)

```
prisma
```

```
model HelpdeskTicket {
 id          String      @id @default(uuid())
 title       String
 description String
 category    TicketCategory
 priority    TicketPriority @default(LOW)
 status      TicketStatus  @default(OPEN)
 raisedBy    String
 assignedTo  String?
 resolution  String?
 resolvedAt  DateTime?
 raiser      User        @relation("TicketRaiser", fields: [raisedBy], references: [id])
 assignee    User?       @relation("TicketAssignee", fields: [assignedTo], references: [id])
 createdAt   DateTime    @default(now())
 updatedAt   DateTime    @updatedAt
}

enum TicketCategory { IT HR ADMIN OTHER }
enum TicketPriority { LOW MEDIUM HIGH URGENT }
enum TicketStatus { OPEN IN_PROGRESS RESOLVED CLOSED }
```

STEP C2 — Helpdesk Backend

FILE: backend/src/services/helpdesk.service.ts

`getMyTickets(userId, query)`:
Own tickets, paginated, filter by status

`getAllTickets(query)`:
HR/Admin: all tickets with raiser info, filter by status/category/priority/assignedTo

`createTicket(userId, data)`:
Create ticket
Notify HR/Admin: "New {priority} ticket: {title} from {employee}"

`assignTicket(ticketId, assignedTo, assignerId)`:
Update assignedTo, status=IN_PROGRESS
Notify assignee

`resolveTicket(ticketId, userId, resolution)`:
Update status=RESOLVED, resolution, resolvedAt=now
Notify ticket raiser

`closeTicket(ticketId)`:
Update status=CLOSED

`getTicketStats()`:
Count by status, count by category, average resolution time

FILE: backend/src/controllers/helpdesk.controller.ts

FILE: backend/src/routes/helpdesk.routes.ts

```
□
```

```
GET   /api/tickets          → getAllTickets (HR/Admin)
GET   /api/tickets/my       → getMyTickets (authenticated)
GET   /api/tickets/stats    → getTicketStats (HR/Admin)
POST  /api/tickets          → createTicket (authenticated)
GET   /api/tickets/:id      → getTicket (authenticated)
PUT   /api/tickets/:id/assign  → assignTicket (HR/Admin)
PUT   /api/tickets/:id/resolve → resolveTicket (HR/Admin)
PUT   /api/tickets/:id/close   → closeTicket (HR/Admin)
```

STEP C3 — Helpdesk Frontend

FILE: frontend/src/api/helpdesk.api.ts

FILE: frontend/src/pages/helpdesk/HelpdeskPage.tsx

Two views:
Employee/Intern view:
PageHeader: "Helpdesk" + "Raise Ticket" button
StatCards: Open, In Progress, Resolved
My Tickets table: Title, Category badge, Priority badge, Status badge, Created, Actions (View detail)
"Raise Ticket" → opens TicketFormModal
Click ticket row → TicketDetailModal
HR/Admin view:
PageHeader: "Helpdesk Management"
StatCards: Open, In Progress, Resolved, Avg Resolution Time
Tabs:
Open Tickets (urgent + high priority highlighted in red/orange)
All Tickets: full filterable table
Stats: bar chart by category, by priority
Table columns: Ticket ID, Title, Category, Priority, Raised By, Assigned To, Status, Created, Actions
Actions: Assign to self, Resolve (with resolution textarea), Close

FILE: frontend/src/components/helpdesk/TicketFormModal.tsx

Form: Title, Description, Category dropdown (IT/HR/Admin/Other), Priority dropdown.

FILE: frontend/src/components/helpdesk/TicketDetailModal.tsx

Shows full ticket: all fields, resolution (if resolved), assigned to, timeline of status changes.

STEP D — Update Admin Dashboard

Update backend/src/services/dashboard.service.ts

Fill in `openTickets` count from HelpdeskTicket where status=OPEN
Fill in `pendingExpenseClaims` count

Update frontend/src/pages/dashboard/AdminDashboardPage.tsx

Connect openTickets StatCard with real data
Add small Announcements section at bottom (latest 2)

AFTER ALL FILES ARE COMPLETE

Write this exactly: "✅ PHASE 7 COMPLETE — Asset Management, Expense Claims, Announcements, and Helpdesk ticketing system are all ready. Proceed to Phase 8 for Reports, Analytics & Settings."